RECIPEHUB SMART RECIPE DISCOVERY WEBSITE: A RESPONSIVE MULTI-PAGE RECIPE DISCOVERY APPLICATION USING HTML5, CSS3 AND VANILLA JAVASCRIPT

CS23531 - MINI-PROJECT REPORT

Submitted by

REVATHI YUVASHREE KJ (241001198)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY
RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled RECIPEHUB SMART RECIPE DISCOVERY WEBSITE RESPONSIVE MULTI-PAGE RECIPE DISCOVERY APPLICATION USING HTML5, CSS3 AND JAVASCRIPT is the bonafide work of REVATHI YUVASHREE KJ (241001198), who carried out the work during the Mini project under my supervision. Certified further that, based on the task given by the training organization, the project was completed under their supervision.

SIGNATURE OF THE CLASS IN-CHARGE

SIGNATURE OF THE REVIEWER

SIGNATURE OF THE HOD

Dr. P. Neelaveni
Head of the Department
Department of Information Technology
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

ACKNOWLEDGEMENT

First, I thank the Almighty for giving me the strength and confidence to complete this mini-project successfully. I express my sincere gratitude to the management of Rajalakshmi Engineering College for providing the facilities and learning environment required to carry out this work.

I extend my heartfelt thanks to our Principal, the Head of the Department of Information Technology, Dr. P. Neelaveni, and the faculty members of the department for their encouragement, guidance and support throughout the project.

I also thank my project guide, classmates, friends and family members for their valuable suggestions, motivation and support during the design, development and documentation of RecipeHub.

REVATHI YUVASHREE KJ

TABLE OF CONTENTS

S.NO	CONTENTS	PAGE NO
—	Abstract	5
1	Introduction	6
2	Literature Review and Background Study	7
3	System Requirements	8
4	System Design and Methodology	10
5	Technology Stack	14
6	Implementation	15
7	Results and Discussion	19
8	Challenges and Issues Faced	20
9	Testing	21
—	Appendix A — HTML Markup Excerpts	23
—	Appendix B — CSS Design Tokens	23
—	Appendix C — JavaScript Interaction Logic	24
10	Conclusion and Future Scope	25
11	References	26
12	System Usage / Use Case Description	26
13	User Manual — Running the Application	27
14	Glossary of Terms	28
—	SDG Mapping	28

Abstract

RecipeHub is a responsive, multi-page recipe discovery website developed as a front-end mini-project. It helps visitors explore a curated collection of twelve recipes, search by recipe name or ingredient, filter by meal category, and discover recipes based on ingredients available in their kitchen. The application also provides recipe details, adjustable serving quantities, favourites, browser-based ratings, recently viewed recipes, a shopping list, cooking mode, a countdown timer and print-friendly recipe information.

The project is implemented using HTML5, CSS3 and Vanilla JavaScript (ES6+) without a server-side framework, database or build tool. The recipe catalogue is maintained centrally in a JavaScript data array and rendered dynamically on the Home and Explore pages. The three HTML pages — Home, Explore Recipes and About — share one stylesheet and one JavaScript file. URL query parameters transfer a search term from the Home page to the Explore page and can also identify a selected recipe.

LocalStorage is used to retain favourites, ratings, the current browser's rating for each recipe, recently viewed recipe IDs, shopping-list items and the selected colour theme. The application validates stored values and handles malformed data to keep the interface usable. As a client-side demonstration, this stored information remains in the visitor's browser and is not synchronised across devices or users.

The report describes the motivation, requirements, architecture, data model, implementation, testing approach, limitations and future enhancements of RecipeHub. Static checks and JavaScript logic checks were performed during development; full browser-based user testing remains a manual verification task.

1. Introduction

Food discovery has increasingly moved to digital platforms. People often have ingredients at home but may not know what they can prepare with them. A useful recipe website should therefore do more than display a list of dishes: it should help users search, compare, understand preparation steps, adjust quantities and organise ingredients before cooking.

RecipeHub – Smart Recipe Discovery Website addresses this need through a clean, responsive interface. The Home page introduces the platform and provides recipe search. The Explore Recipes page presents the recipe collection with category filters, sorting and instant search. Its Smart Ingredient Finder compares selected ingredients with the ingredient lists of recipes and reports the matching percentage and missing ingredients. The About page explains the purpose and key features of the application.

A selected recipe can be opened in a details modal that displays its description, cuisine, preparation and cooking times, difficulty, ingredients and step-by-step instructions. Users can change the serving count and see ingredient quantities recalculated. They can also save favourites, rate recipes, add ingredients to a shopping list, open a focused cooking mode, use a countdown timer and print the recipe.

The project deliberately uses only browser-side technologies. HTML5 defines the page structure, CSS3 controls the visual design and responsive layout, and Vanilla JavaScript implements the interactions and data handling. LocalStorage provides persistence for the current browser. There is no account system, cloud database or external recipe API; the project is a self-contained front-end demonstration.

The report documents the background study, requirements, design, technology stack, implementation modules, results, challenges and testing plan. It also identifies limitations and possible future improvements.

1.1 Objectives

• Develop a polished, responsive recipe discovery website with three linked pages.

• Provide a centrally maintained dataset of twelve complete recipes.

• Implement instant search, category filtering and sorting.

• Create an ingredient matching system with normalisation, match percentages and strict matching.

• Provide persistent favourites, ratings, recently viewed recipes and shopping-list features.

• Improve cooking usability through serving adjustment, cooking mode, timer and print support.

• Maintain readable, beginner-friendly code using only HTML5, CSS3 and Vanilla JavaScript.

2. Literature Review and Background Study

2.1 Digital Recipe Discovery

Recipe websites commonly organise content through searchable collections, categories and detailed recipe pages. A useful recipe entry includes ingredients, quantities, preparation time, cooking time, serving size and ordered instructions. These details help a visitor decide whether a recipe is suitable before beginning preparation. RecipeHub applies these familiar patterns to a compact, data-driven collection.

2.2 Responsive Web Design

Responsive web design allows one website to adapt to different screen sizes using flexible layouts, CSS Grid, Flexbox, fluid dimensions and media queries. A recipe card grid can show multiple cards on a desktop and fewer cards on a phone without requiring separate websites. RecipeHub uses CSS Grid and Flexbox with responsive breakpoints to keep navigation, cards, forms and modal content usable on desktop, tablet and mobile screens.

2.3 Multi-Page Websites and Shared Resources

A multi-page website separates distinct tasks into individual HTML documents connected through hyperlinks. RecipeHub uses index.html for the Home page, recipe.html for recipe exploration and about.html for project information. A shared style.css file keeps visual rules consistent, while script.js contains the central recipe data and shared interaction logic. URLSearchParams reads search and recipe identifiers from the query string.

2.4 Browser Storage and LocalStorage

The Web Storage API provides localStorage, a browser key–value store that persists string data for an origin across reloads and browser restarts. Structured values are converted to JSON text with JSON.stringify and restored with JSON.parse. RecipeHub uses separate keys for favourites, ratings, user ratings, recently viewed recipes, shopping-list items and theme preference. This approach is suitable for a front-end demonstration, but the values are limited to the current browser and can be cleared by the visitor.

2.5 Ingredient Matching and Normalisation

Ingredient matching requires consistent comparison. The same ingredient may be written in singular or plural form, or by a regional or alternate name. Normalisation converts known aliases into a common term before comparison. For example, potatoes are mapped to potato, capsicum to bell pepper, maida and all-purpose flour to flour, and curd or yogurt to yoghurt. The finder then calculates how many recipe ingredients are available and derives a match percentage.

2.6 Usability and Accessibility

Usable interfaces provide clear labels, meaningful button names, visible keyboard focus, readable contrast, semantic page structure and helpful feedback. A modal should support keyboard dismissal and avoid accidental closure from clicks inside its content. Responsive navigation should remain reachable on small screens. RecipeHub incorporates semantic HTML, accessible labels and modal controls, with further assistive-technology evaluation recommended as future testing.

2.7 Summary

The background study supports a data-driven recipe catalogue, combined search and filtering, normalised ingredient comparison, browser storage for personal preferences and responsive design. These principles guide the architecture and implementation described in the next chapters.

3. System Requirements

3.1 Hardware Requirements

• A desktop, laptop, tablet or smartphone capable of running a modern web browser.

• A standard display and keyboard or touch input device.

• Approximately 4 GB RAM is sufficient for normal development and browser testing.

• Internet access is required for externally hosted Unsplash food photographs; core recipe data and interactions are stored locally in the project.

3.2 Software Requirements

• A current version of Google Chrome, Microsoft Edge or Mozilla Firefox.

• Visual Studio Code or another text editor for editing the project files.

• VS Code Live Server is recommended for running the website locally.

• Browser developer tools for inspecting the DOM, JavaScript console and LocalStorage.

3.3 Technologies and External Resources

Resource	Purpose
HTML5	Semantic structure for the Home, Explore Recipes and About pages.
CSS3	Shared visual design, CSS variables, Grid, Flexbox, responsive media queries and print rules.
Vanilla JavaScript (ES6+)	Recipe data, rendering, search, filtering, ingredient matching, modal behaviour and browser storage.
Browser LocalStorage	Persistence for favourites, ratings, recently viewed recipes, shopping list and theme.
Unsplash image URLs	Food photography used as recipe and presentation imagery; image loading depends on network access.
VS Code Live Server	Local development server for opening and testing the multi-page project.

3.4 Functional Requirements

• Display a Home page with navigation, a hero section, search and featured recipe content.

• Display twelve recipes on the Explore Recipes page and support instant search by name, ingredient, category, cuisine and description.

• Filter recipes by All Recipes, Breakfast, Lunch, Dinner, Snacks and Desserts; combine category and search filters.

• Match selected ingredients against recipe ingredients, calculate match percentages, show missing ingredients and support strict matching.

• Allow users to open recipe details, adjust servings and view ingredient quantities and instructions.

• Allow favourites and ratings to be added, changed and retained in LocalStorage.

• Provide recently viewed recipes, a shopping list, cooking mode, countdown timer and print option.

• Provide light and dark themes and responsive mobile navigation.

3.5 Non-Functional Requirements

• Responsive: usable on phone, tablet and desktop widths without horizontal overflow.

• Maintainable: one central recipe dataset and shared CSS/JavaScript files.

• Portable: runs as static files without a backend, database, package manager or compilation.

- Reliable: validate LocalStorage values and avoid accessing page-specific elements when they are absent.
- Accessible: semantic elements, labelled inputs, keyboard focus and clear control names.

4. System Design and Methodology

4.1 Development Approach

The website was developed in stages. First, the page structure and visual identity were planned. Second, the central recipe dataset and recipe-card rendering were implemented. Third, search, categories, ingredient matching and recipe details were added. The final stage covered LocalStorage features, cooking utilities, responsive styling, accessibility considerations and code checks.

File	Responsibility
index.html	Home page with hero, search, featured recipes and navigation.
recipe.html	Explore page with recipe catalogue, categories, search, sorting, favourites, ingredient finder and recipe utilities.
about.html	About page describing the purpose and features of RecipeHub.
style.css	Shared design tokens, page layouts, cards, controls, modal, responsive and print styles.
script.js	Central recipe dataset, page-aware initialisation, search, filters, ingredient matching, modal, ratings, favourites, LocalStorage and cooking tools.

Table 4.1 — Project files and responsibilities

4.2 Information Architecture

Page	Address	Purpose
Home	index.html	Introduction, recipe search, featured recipes and links to Explore and About.
Explore Recipes	recipe.html	Full recipe catalogue, search, category filters, sorting and Smart Ingredient Finder.
About	about.html	Project description, feature explanations and call to action.
Recipe details	recipe.html?recipe=	Selected recipe details are opened in the modal using the recipe identifier.

Table 4.2 — Pages and navigation

4.3 Visual Design System

The interface uses a warm cream background, terracotta accent colour, dark charcoal text, beige secondary surfaces, subtle borders and soft shadows. Rounded recipe cards and generous spacing make the catalogue approachable. CSS custom properties centralise colours, spacing, shadows and border radii so that the visual system can be adjusted consistently. Light and dark themes are controlled through a saved theme preference.

Design element	Purpose
Warm cream / off-white background	Creates a welcoming food-oriented canvas.
Terracotta / burnt-orange accent	Highlights primary actions, active filters and brand elements.
Dark charcoal text	Maintains readable contrast on light surfaces.
Rounded cards and subtle shadows	Separates recipe content without a heavy visual border.
CSS variables	Centralise reusable colour, spacing, shadow and radius values.
Responsive Grid and Flexbox	Rearrange recipe cards, navigation and content at different widths.

Table 4.3 — Main design decisions

4.4 Recipe Data Model

The recipe collection is stored once in a JavaScript array named RECIPES. Each object contains a unique identifier, recipe name, category, cuisine, description, image URL, preparation time, cooking time, difficulty, default serving count, ingredient records, instructions and dietary tags. The same dataset supplies cards, the Home page featured section, the modal, the ingredient finder and recently viewed items.

Field	Type	Purpose
id	Number / string	Unique key for links and LocalStorage references.
name, category, cuisine	String	Display information and search/filter matching.
description, image	String	Recipe summary and food photograph source.
prepTime, cookTime	Number	Preparation and cooking duration.
difficulty, servings	String / number	Skill level and default serving quantity.
ingredients	Array of objects	Ingredient name, quantity and unit used by matching and serving calculator.
instructions	Array of strings	Ordered cooking steps used by details and cooking mode. Explicit
dietaryTags	Array of strings	dietary labels, where supported by recipe data.

Table 4.4 — Recipe object fields

4. System Design and Methodology (Continued)

4.5 Application Flow

When a page loads, the browser reads its HTML and shared stylesheet, then executes script.js. The script checks which page-specific elements are present before attaching listeners or rendering content. Shared functions load saved preferences and update common navigation elements. On the Explore page, the recipe dataset is rendered and filtered based on the current search and category. When a recipe is opened, the selected recipe is recorded in the recently viewed list and its details are placed in the modal.

```
HOME PAGE
index.html
   | search term in URL query parameter
   v
EXPLORE PAGE
recipe.html
   |-- Load central RECIPES array
   |-- Apply search + category + sort
   |-- Render recipe cards
   |-- Ingredient Finder compares selected ingredients
   |-- Open selected recipe modal
   v
RECIPE UTILITIES
   |-- Serving calculator
   |-- Favourite and rating controls
   |-- Shopping list
   |-- Cooking mode and timer
   |-- Print recipe
   v
BROWSER LOCALSTORAGE
   favourites | ratings | user ratings
   recently viewed | shopping list | theme
```

Fig 4.1 — Text representation of the RecipeHub application flow

4.6 Ingredient Matching Method

The finder first normalises the user's selected ingredients and the ingredient names in each recipe. Known aliases are converted to canonical names. The system then identifies the intersection between the selected ingredients and the recipe's required ingredient list. Match percentage is calculated from the count of matched ingredients divided by the count of required ingredients, multiplied by 100. Results are sorted from highest to lowest match percentage.

When strict matching is enabled, only recipes for which all required ingredients are selected are displayed. When strict matching is disabled, recipes can be shown with missing ingredient information so that the user can decide whether to continue. The interface must not label a partially matched recipe as fully achievable.

```
matchPercentage = (matchedCount / requiredCount) * 100

Strict mode:
 show recipe only when matchedCount === requiredCount

Normal mode:
 show matching recipes with percentage
show missing ingredient names
```

4.7 LocalStorage Data Design

Storage key / data group	Stored information
Favourites	Array of recipe IDs saved by the visitor.
Recipe ratings	Aggregated rating values and rating counts held in this browser.
User ratings	The current browser's selected rating per recipe; changing a rating updates the existing vote.
Recently viewed	Unique recipe IDs ordered newest first, limited to a small recent-history list.
Shopping list	Ingredient items with purchased state and identifiers.
Theme preference	Light or dark theme selection.

Table 4.5 — Browser storage design

All storage reads are validated before use. Invalid JSON or unexpected data types are handled safely so that one damaged value does not prevent the rest of the application from loading. Since LocalStorage is browser-specific, data is not shared with other users or devices.

4.8 Main Interaction Rules

• A recipe card opens its details modal; clicks on the heart or rating controls are kept separate from the card-opening action.

• The serving calculator uses the original serving count as its baseline, keeps the result between 1 and 20, and scales ingredient quantities proportionally.

• Changing a user's rating updates the existing browser vote instead of adding another vote.

• Recently viewed entries are unique and the newest recipe appears first.

• Shopping-list entries avoid unnecessary duplicates and support purchased, remove and clear actions.

• The modal can close using its close control, backdrop or Escape key, and background scrolling is disabled while it is open.

5. Technology Stack

• **Markup:** HTML5 semantic elements such as header, nav, main, section, article, form and footer are used to structure the three pages.

• **Styling:** CSS3 custom properties, Grid, Flexbox, transitions, media queries, focus styles and print rules are maintained in style.css.

• **Scripting:** Vanilla JavaScript (ES6+) implements functions, arrays, objects, event listeners, template rendering, URLSearchParams, JSON and LocalStorage.

• **Image resources:** Food photographs are referenced from Unsplash. The site does not depend on external recipe APIs.

• **Persistence:** Browser LocalStorage stores personal preferences and application data for the current browser.

• **Development:** Visual Studio Code and Live Server provide a simple local development workflow.

5.1 Why Vanilla JavaScript?

Vanilla JavaScript satisfies the project restriction and demonstrates the browser's native capabilities directly. It avoids framework setup, package installation and compilation. The code can be opened and studied using a standard editor, making it suitable for a beginner-friendly college mini-project.

5.2 Why LocalStorage?

LocalStorage is sufficient for a single-browser demonstration of favourites, ratings, recently viewed recipes, theme selection and shopping-list state. It allows the user to reload the page without losing these values. It is not a replacement for a database or secure server-side account storage.

6. Implementation

6.1 HTML Page Structure

Each HTML page links to the same style.css and script.js files. The Home page contains the hero area, search form and featured content. The Explore page contains the category toolbar, search controls, sort options, recipe grid, ingredient finder, favourites, recently viewed section and supporting recipe utilities. The About page contains a project introduction and feature descriptions. JavaScript initialises only the modules whose required elements exist on the current page.

6.2 Central Recipe Dataset

The RECIPES array is the single source of recipe information. It includes the twelve recipes specified for the project: Creamy Garden Pasta, Fresh Rainbow Bowl, Classic Homemade Pizza, Masala Dosa, Fluffy Pancakes, Chocolate Brownies, Veggie Sandwich, Creamy Tomato Soup, Berry Smoothie, Vegetable Fried Rice, Chocolate Cake and Crispy Samosa. Each recipe has its own ingredients, quantities, instructions, preparation/cooking details and supported dietary tags.

No.	Recipe	Category
1	Creamy Garden Pasta	Dinner
2	Fresh Rainbow Bowl	Lunch
3	Classic Homemade Pizza	Dinner
4	Masala Dosa	Breakfast
5	Fluffy Pancakes	Breakfast
6	Chocolate Brownies	Desserts
7	Veggie Sandwich	Snacks
8	Creamy Tomato Soup	Lunch
9	Berry Smoothie	Breakfast
10	Vegetable Fried Rice	Dinner
11	Chocolate Cake	Desserts
12	Crispy Samosa	Snacks

Table 6.1 — Recipe catalogue

6.3 Home Page

The Home page introduces RecipeHub through the eyebrow text “YOUR DAILY DOSE OF DELICIOUS” and the heading “Good food. Good mood.” It provides a search field and an Explore Recipes call to action. When the search form is submitted, the query is placed in a URL parameter and the browser navigates to recipe.html. The Explore page reads the parameter and applies it to the catalogue, allowing a search to continue across pages.

6.4 Explore Recipes and Search

The Explore page renders recipe cards from RECIPES. Search is case-insensitive and checks recipe name, ingredient names, category, cuisine and description. Typing updates the result list immediately, while the search button supports explicit submission. A clear-search control restores the unfiltered query state. Category filters work together with search rather than replacing it, and an empty-state message is displayed when no recipe matches.

6.5 Sorting and Dietary Filters

The sorting controls support default order, alphabetical order, shortest cooking time and highest average rating. Rating-based sorting becomes meaningful after ratings have been entered in the current browser. Dietary filters are shown only where the recipe dataset contains explicit dietary tags; suitability is not inferred from the recipe title.

6.6 Smart Ingredient Finder

The ingredient finder provides a text input, add control, quick-add suggestions, selected ingredient chips, remove controls, Clear All and Find My Recipes actions. The suggestions include common items such as potato, tomato, onion, egg, bread, cheese, rice, flour, milk, butter, carrot, cucumber, garlic, peas, chocolate, banana, pasta and oil. Alias normalisation helps match common variations such as potatoes, tomatoes, capsicum, maida, all-purpose flour, curd, yogurt and cooked rice.

The results display matching recipes in descending order of match percentage. In non-strict mode, missing ingredients are identified. In strict mode, only recipes whose required ingredient list is fully covered are displayed. Start Again resets the finder selection and results.

6. Implementation (Continued)

6.7 Recipe Cards and Details Modal

Each recipe card displays a food image, recipe name, cuisine, difficulty, total cooking time, a favourite heart control and a five-star rating interface with the average rating and number of ratings. The card opens the selected recipe in a details modal. Event handling prevents clicks on the heart and rating stars from triggering the card's modal action.

The modal is populated from the selected recipe object rather than from fixed content. It shows a large image, recipe name, cuisine, description, preparation time, cooking time, difficulty, ingredients, quantities and instructions. It also includes favourite, serving calculator, shopping-list, cooking-mode and print actions where provided by the interface.

6.8 Serving Calculator

The serving calculator begins with the recipe's default serving count. The plus and minus controls adjust the serving count within the allowed range of 1 to 20. Ingredient quantities are multiplied by the ratio of the selected serving count to the original serving count. Decimal values are formatted neatly, with common fractions used where practical, so that values such as three-eighths of a cup remain readable.

6.9 Favourites and Ratings

A visitor can toggle the heart control to add or remove a recipe from favourites. The navigation favourite count and all visible recipe copies are updated to reflect the saved state. The Favourites section shows saved recipes or a friendly empty state. The rating interface accepts one to five stars. The current browser's rating for a recipe can be changed; the existing vote is updated rather than counted as an additional rating. A thank-you message confirms the interaction. The project is front-end only. Ratings are stored in the current browser and are not a shared public rating service. A different browser or device has a separate LocalStorage area.

6.10 Recently Viewed Recipes

When a visitor opens a recipe, its ID is added to the recently viewed list. Duplicate IDs are removed, the newest item is placed first and the list is limited to six recipes. The list is restored from LocalStorage when the page is reloaded.

6.11 Shopping List

The shopping list accepts ingredients from a recipe. Items can be checked as purchased, removed individually, cleared when purchased or cleared completely. The list is stored in LocalStorage. Duplicate ingredient entries are avoided where practical to keep the list concise.

6.12 Cooking Mode and Timer

Cooking mode presents one instruction at a time along with the recipe name, current step number and total number of steps. Previous and Next controls move through the instructions. A completion message appears after the final step, and an Exit control returns the visitor to the normal recipe view. The countdown timer supports time input, Start, Pause, Resume and Reset. Timer intervals are cleaned up when no longer needed, and a completion notification is shown when the countdown reaches zero.

6.13 Print, Theme and Navigation

The Print Recipe action invokes the browser print workflow. Print-specific CSS hides navigation, buttons, modal overlays and unrelated sections so that the selected recipe content, ingredients and instructions are prioritised. The theme toggle switches between light and dark modes and saves the preference. On small screens, the navigation menu can be opened and closed; selecting a navigation link closes the menu.

6.14 Accessibility and Performance

- Semantic HTML and labels are used for page structure and form controls.
- Meaningful accessible names are provided for interactive buttons and images.
- Keyboard focus is visible, and modal dismissal supports Escape.

- Noncritical images use lazy loading where applicable.
- Event delegation and page-aware initialisation reduce repeated listeners and missing-element errors.
- Reduced-motion preferences are considered in the stylesheet.

7. Results and Discussion

7.1 Overview of the Delivered Website

The delivered project consists of three HTML pages, one shared stylesheet and one shared JavaScript file. The recipe catalogue contains twelve data-driven entries. The website is designed to provide a connected user journey from homepage search to recipe exploration, ingredient matching and cooking support.

7.2 Home Page

The Home page presents the RecipeHub identity, the headline “Good food. Good mood.”, a short description and a search field. A visitor can enter a recipe or ingredient and continue to the Explore page with the query preserved in the URL. Featured recipe content is generated from the shared recipe dataset rather than being maintained as a separate copy.

7.3 Explore Recipes Page

The Explore page displays the twelve recipe cards. Category buttons narrow the list by meal category. The search field filters as the visitor types, and the category and search conditions are applied together. Sorting changes the order of the matching recipes. If no recipe satisfies the active conditions, a friendly empty state is displayed.

7.4 Smart Ingredient Finder

The finder compares selected ingredients against the recipe ingredient data. Matching recipes are ordered by percentage, and missing ingredients are visible when strict matching is disabled. Strict mode removes partially matched recipes from the result set. Alias handling improves the chance that familiar ingredient variations are recognised.

7.5 Recipe Modal and Cooking Support

The recipe modal displays data for the recipe selected by the visitor. Ingredient quantities respond to serving changes, while the cooking mode focuses on one instruction at a time. The shopping list can be populated from recipe ingredients, and the timer supports common countdown controls. Print styles are intended to provide a simplified paper-friendly recipe view.

7.6 Persistent Features

Favourites, ratings, recently viewed recipes, shopping-list items and theme selection are saved in LocalStorage. On reload, the website reads the saved values and restores the corresponding interface state. The storage design is local to the browser and does not imply user accounts or cloud synchronisation.

7.7 Development Checks and Verification Status

During development, the JavaScript file was checked for syntax validity using a Node syntax check. Static checks were also used to compare the DOM IDs referenced by the script with IDs present in the HTML, detect duplicate IDs, verify unique recipe identifiers and exercise ingredient normalisation, matching, rating-update and malformed-storage handling. These checks are not equivalent to a full browser test. The complete interface still requires manual execution in a browser using Live Server.

Aspect	Intended outcome / development check
Page structure	Three linked pages share style.css and script.js.
Recipe data	Twelve recipes are held centrally with unique identifiers.
Search and categories	Search query and category conditions are combined.
Ingredient finder	Alias normalisation, match percentage and strict mode are implemented in logic.
LocalStorage	Separate saved values are validated before use.
JavaScript syntax	Node syntax check completed during development.
Browser UI verification	Not claimed as completed; manual browser testing is required.

Table 7.1 — Development checks and verification status

8. Challenges and Issues Faced

8.1 Multi-Page Script Safety

A shared JavaScript file runs on pages that do not contain the same elements. Directly accessing an element that is absent can cause a null-reference error. The implementation addresses this by checking for required elements before initialising page-specific features and by keeping shared functions separate from page-specific setup.

8.2 Ingredient Name Variations

Users may enter plural names or alternate names for the same ingredient. Direct string comparison would miss valid matches. The normalisation step maps known aliases to a canonical ingredient name before comparing the selected list with recipe data.

8.3 Keeping Multiple Recipe Views Consistent

A recipe may appear in the main catalogue, favourites, finder results, recently viewed content and the details modal. A change to favourite or rating state should be reflected in every visible copy. The shared recipe IDs and common state functions provide a consistent reference for these updates.

8.4 Accurate Ingredient Matching

A recipe should not be presented as fully achievable when required ingredients are missing. The finder calculates the match percentage and separately records missing ingredients. Strict mode displays only complete matches, avoiding an inaccurate result label.

8.5 Persistent Data and Malformed Storage

LocalStorage contains strings and may contain invalid JSON or unexpected values. Parsing without validation can interrupt page initialisation. Storage-reading functions therefore use guarded parsing and validate the resulting structure before the application relies on it.

8.6 Quantity Formatting

Scaling ingredients can produce decimal quantities that are awkward for cooking. The serving calculator formats quantities neatly and uses common fractions where practical. A development check identified a quantity-formatting issue in which 0.375 cup appeared as 0.4 cup; the formatting logic was corrected to display three-eighths where supported.

8.7 External Image Availability

Food photographs are loaded from Unsplash image URLs. External images require a network connection and an individual image URL may become unavailable. A fallback presentation is used when an image cannot be loaded. Replacing or verifying image URLs is a maintenance task.

8.8 Front-End Limitations

- Favourites, ratings and shopping-list data are stored only in the current browser.
- There is no user authentication, cloud database, synchronisation or shared rating service.
- The recipe catalogue is a fixed JavaScript dataset and must be edited in the source to add recipes.
- The countdown timer runs in the active browser page and is not a server-side timer.
- A complete browser, cross-device and assistive-technology test has not been claimed.

9. Testing

Testing is required to confirm that each page loads, navigation is correct, recipe features behave as expected and saved values survive a reload. The following table is a manual test plan for the final project. The JavaScript syntax and selected logic checks were completed during development, but each browser interaction should be executed and recorded on the submitted build.

No.	Test case	Action / input	Expected outcome
1	Load Home	Open index.html	Hero, navigation and featured section appear.
2	Home search	Search for "tomato"	Open recipe.html with query preserved and matching results.
3	Explore catalogue	Open recipe.html	Twelve recipe cards are rendered.
4	Category filter	Select Breakfast	Only Breakfast recipes remain.
5	Instant search	Type a recipe or ingredient	Matching cards update without page reload.
6	Combined filters	Select category and enter query	Only results satisfying both conditions remain.
7	Clear search	Use clear-search control	Search term is removed and category remains usable.
8	No results	Enter unmatched text	Friendly empty state is displayed.
9	Sort	Choose alphabetical order	Matching recipes appear alphabetically.
10	Ingredient alias	Enter "potatoes"	Alias normalises to potato for matching.
11	Ingredient matching	Select several ingredients	Results show match percentage and missing items.
12	Strict matching	Enable strict mode	Only recipes with all required ingredients are shown.
13	Favourite	Click a recipe heart	Favourite state and navigation count update.
14	Favourite persistence	Reload after favouriting	Saved favourite remains visible.
15	Open modal	Click a recipe card	Correct selected recipe details are displayed.
16	Modal close	Use close, backdrop and Escape	Modal closes; inner modal clicks do not close it.
17	Serving calculator	Change servings	Ingredient quantities scale proportionally within 1–20.
18	Rating	Select a star	Rating and count update and confirmation appears.
19	Change rating	Select a different star	Existing browser rating is updated, not duplicated.
20	Recently viewed	Open multiple recipes	Newest unique recipe appears first; list is limited.
21	Shopping list	Add recipe ingredients	Items appear and persist after reload.
22	Purchased item	Check a shopping-list item	Purchased state is shown.
23	Cooking mode	Start and move through steps	Previous/Next and completion state work.
24	Timer	Start, pause, resume and reset	Countdown responds to each control.
25	Print recipe	Choose Print Recipe	Only relevant recipe content is prioritised.
26	Dark mode	Toggle theme and reload	Selected theme is restored.
27	Mobile navigation	Use narrow viewport and menu	Menu opens, closes and closes after navigation.
28	Malformed storage	Insert invalid JSON in a storage key	Application recovers without a fatal page error.
29	About page	Open about.html	About content and shared navigation load.
30	Responsive layout	Test phone, tablet and desktop widths	Content adapts without horizontal overflow.

Table 9.1 — Manual test cases and expected outcomes

9.1 Test Evidence and Reporting

For final submission, execute the test cases in a browser using VS Code Live Server. Record the actual result as Pass or Fail, note the browser and viewport size, and capture screenshots for important features such as search, ingredient matching, the recipe modal, LocalStorage persistence and mobile navigation. Do not mark a browser test as passed until it has been performed on the running website.

Appendix A — HTML Markup Excerpts

The following excerpts illustrate the structure of the project pages. They are representative examples; the complete markup is contained in the submitted HTML files.

A.1 Shared stylesheet and script links

```
<link rel="stylesheet" href="style.css">
<script src="script.js" defer></script>
```

A.2 Home page search form

```
<form id="home-search-form">
 <label for="home-search">Find a recipe</label>
<input id="home-search" type="search"
         placeholder="Search recipes or
 <button type="submit">Search</button>
</form>
```

A.3 Explore page recipe and finder regions

```
<section aria-labelledby="recipes-heading">
 <h1 id="recipes-heading">Explore Recipes</h1>
<div id="category-buttons"></div>
 <input id="search-input" type="search">
 <div id="recipe-grid"></div>
</section>

<section aria-labelledby="finder-heading">
 <h2 id="finder-heading">Smart Ingredient Finder</h2>
<input id="ingredient-input" type="text">
 <button id="add-ingredient">Add</button>
 <div id="selected-ingredients"></div>
 <button id="find-recipes">Find My Recipes</button>
<div id="finder-results"></div>
</section>
```

Appendix B — CSS Design Tokens and Responsive Rules

B.1 Design tokens

```
:root {
 --color-bg: #fbf7f0;
 --color-primary: #c66a45;
 --color-text: #262626;
 --color-surface: #ffffff;
 --color-border: #e7ded4;
 --radius-card: 18px;
 --shadow-card: 0 8px 24px rgba(38, 38, 38, .08);
}
```

B.2 Responsive card layout

```
.recipe-grid {
 display: grid;
 grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 1.5rem;
}
@media (max-width: 900px) {
 .recipe-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
@media (max-width: 600px) {
.recipe-grid {
    grid-template-columns: 1fr;
  }
}
```

The actual stylesheet also contains the detailed component styles, dark-theme variables, focus states, modal layout, reduced-motion handling and print-specific rules.

Appendix C — JavaScript Interaction Logic

C.1 Safe LocalStorage reading pattern

```
function readStoredArray(key) {
try {
    const value = JSON.parse(localStorage.getItem(key));
    return Array.isArray(value) ? value : [];
 } catch (error) {
    return [];
  }
}
```

C.2 Ingredient alias normalisation

```
constingredientAliases = {
 potatoes: "potato",
 tomatoes: "tomato",
   capsicum:   "bell   pepper",
maida: "flour",
  "all-purpose  flour":  "flour",
curd: "yoghurt",
 yogurt: "yoghurt",
 "cooked rice": "rice"
};
```

C.3 Match percentage

```
const matchPercentage=
 requiredIngredients.length === 0
    ? 0
    : Math.round(
        matchedIngredients.length /
        requiredIngredients.length * 100
      );
```

These excerpts explain the core approach. The full script contains the complete recipe dataset and the page-specific event handling, rendering, rating, modal, serving, shopping-list, timer and cooking-mode functions.

10. Conclusion and Future Scope

10.1 Conclusion

RecipeHub demonstrates how a polished, interactive recipe discovery website can be developed using HTML5, CSS3 and Vanilla JavaScript without a server-side framework or database. The three-page structure, shared stylesheet and shared script provide a consistent experience. A central dataset of twelve recipes drives the Home page content, Explore catalogue, search, category filters, recipe details and Smart Ingredient Finder.

The project combines recipe discovery with practical cooking support. Users can identify recipes from available ingredients, view missing items, adjust serving quantities, save favourites, rate recipes, review recently viewed items, build a shopping list, follow instructions in cooking mode, use a timer and print recipe details. LocalStorage provides persistence for the current browser, while defensive validation helps the application recover from invalid stored data. The development process provided practical experience with semantic HTML, responsive CSS, DOM manipulation, JavaScript arrays and objects, event handling, URL query parameters, JSON, browser storage, accessibility considerations and testing. It also highlighted the importance of keeping shared data centralised and checking page-specific elements before attaching event listeners.

10.2 Known Limitations

• There is no backend, user authentication, cloud database or cross-device synchronisation.

• Favourites, ratings and shopping-list information are local to the current browser.

• The recipe dataset is maintained in JavaScript and requires source-code changes to add or edit recipes.

• Food photographs are external Unsplash resources and require network access to load.

• The rating system is a local demonstration and does not represent ratings from a public user community.

• A full browser-based test, cross-browser audit and screen-reader evaluation remain to be completed.

10.3 Future Enhancements

• Add an optional backend and database to manage recipes and user accounts.

• Provide secure account-based synchronisation of favourites, ratings and shopping lists.

• Allow users to submit recipes, photos, comments and reviews with validation and moderation.

• Add advanced dietary, allergen and nutrition filters backed by verified recipe data.

• Add meal planning, weekly shopping-list generation and recipe recommendations.

• Improve offline support and image optimisation, and perform automated accessibility and performance audits.

11. References

• MDN Web Docs — HTML: HyperText Markup Language. https://developer.mozilla.org/en-US/docs/Web/HTML

• MDN Web Docs — CSS: Cascading Style Sheets. https://developer.mozilla.org/en-US/docs/Web/CSS

• MDN Web Docs — JavaScript Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

• MDN Web Docs — Window.localStorage. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage • MDN Web Docs — URLSearchParams. https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams

• WHATWG — HTML Living Standard. https://html.spec.whatwg.org/

• W3C — Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

• Unsplash — Foodphotographyresources.https://unsplash.com/

12. System Usage / Use Case Description

The primary actor is the Visitor. The visitor opens the Home page, searches or browses the recipe collection, uses the ingredient finder, opens recipe details and optionally saves preferences or uses the cooking utilities. The project does not include a separate administrator or authenticated user role.

Use case	Description
Browse featured recipes	Visitor opens Home and views the introduction and featured recipes.
Search recipes	Visitor enters a recipe or ingredient keyword; Explore displays matching recipes.
Filter by category	Visitor selects a meal category and the recipe grid updates.
Find recipes by ingredients	Visitor adds available ingredients and reviews match percentages and missing ingredients.
View recipe details	Visitor opens a recipe modal to view its description, timings, ingredients and instructions.
Adjust servings	Visitor changes the serving count and sees recalculated quantities.
Manage favourites and ratings	Visitor saves or removes favourites and submits or changes a rating.
Manage shopping list	Visitor adds recipe ingredients, marks purchases and removes items.
Use cooking mode and timer	Visitor follows instructions step by step and runs a countdown.
Change theme / print	Visitor selects a theme or prints recipe information.

Table 12.1 — RecipeHub use cases

13. User Manual — Running the Application

Step 1: Prepare the project files

Create one folder named RecipeHub and place the five project files inside it: index.html, recipe.html, about.html, style.css and script.js. Keep the filenames unchanged so that the page links and shared resource paths continue to work.

Step 2: Project structure

```
RecipeHub/
index.html          Home page
recipe.html         Explore Recipes page
about.html          About page
style.css           Shared stylesheet
script.js           Recipe data and application logic
```

Step 3: Open the website

Open the RecipeHub folder in Visual Studio Code. If Live Server is installed, right-click index.html and choose “Open with Live Server”. The website opens in the default browser. A modern browser is recommended. Internet connectivity may be needed for externally hosted food images.

Step 4: Use the website

- Use the navigation bar to move between Home, Explore Recipes and About.
- Enter a recipe name or ingredient on the Home page to search the catalogue.
- On Explore Recipes, select a category, type a keyword or change the sorting option.
- Use Smart Ingredient Finder to add ingredients, select strict matching if required and find matching recipes.
- Open a recipe card to view details, change servings, add ingredients to the shopping list or start cooking mode.
- Use the heart and star controls to manage favourites and ratings.
- Use the timer controls while cooking, and choose Print Recipe to print the selected recipe.
- Toggle the theme control to switch between light and dark appearance.

Step 5: Reset saved data

Saved favourites, ratings, recently viewed recipes, shopping-list items and theme preference are stored in the browser's LocalStorage. To start with a clean state, clear the site's LocalStorage through the browser developer tools. This action removes the locally saved application data for the site.

14. Glossary of Terms

Term	Definition
HTML5	Markup language used to structure the website and its semantic page elements.
CSS3	Style language used for colours, typography, layout, responsiveness and print presentation.
Vanilla JavaScript	JavaScript written without external frameworks or libraries.
DOM	Document Object Model; the browser representation of the page that JavaScript can read and update.
LocalStorage	Browser storage that retains string values for a website between page reloads.
JSON	JavaScript Object Notation; text format used to serialise structured browser data.
URL query parameter	A value after the question mark in a URL, used to transfer search terms or recipe identifiers.
Responsive design	Design approach that adapts page layout to different screen sizes.
CSS Grid / Flexbox	CSS layout systems used to arrange cards and interface components.
Ingredient normalisation	Converting alternate ingredient spellings or names to a common comparison value.
Match percentage	Proportion of a recipe's required ingredients present in the visitor's selected ingredients.
Strict matching	Finder mode that displays only recipes whose required ingredients are all selected.
Accessibility	Designing the interface so people with different abilities and input methods can use it.
Unsplash	External image source usedforfoodphotographyinthe project.

SDG Mapping

Project Title: RECIPEHUB – SMART RECIPE DISCOVERY WEBSITE

SDG (No. & Theme)	Activity addressing SDG theme
SDG 3 – Good Health and Well-being	Organises recipe information and ingredient details that can help visitors plan home-cooked meals. The site does not provide medical or nutrition advice.
SDG 4 – Quality Education	Provides a practical learning project demonstrating web development, data handling, responsive design and interactive user-interface concepts.
SDG 9 – Industry, Innovation and Infrastructure	Demonstrates a lightweight digital application built with standard web technologies and no server-side dependency.
SDG 12 – Responsible Consumption and Production	The ingredient finder and shopping list can help users plan ingredient use and prepare a more organised shopping list.

Table — Sustainable Development Goal mapping